

Lecture 4 — File Formats: CSV vs Parquet and Columnar Techniques

Running example data (same 5 employees used across both file-format and database diagrams)

id=1 Alice (Developer, active) · id=2 Bob (Designer, inactive) · id=3 Charlie (Manager, active) · id=4 Dana (Developer, inactive) · id=5 Eve (Designer, active)

1. Same data, two physical layouts

CSV — ROW-major (one row at a time)

```
id, name, role, active, joined_date
1, Alice, Developer, true, 2026-05-12
2, Bob, Designer, false, 2024-11-03
3, Charlie, Manager, true, 2026-05-20
4, Dana, Developer, false, 2025-08-15
5, Eve, Designer, true, 2026-02-28
```

Disk layout:
[row 1][row 2][row 3][row 4][row 5]

To compute "avg active rate" the reader scans EVERY byte of every row to reach the one column it cares about.

Parquet — COLUMN-major (one column at a time)

```
id : [ 1, 2, 3, 4, 5 ]
name : [ Alice, Bob, Charlie, Dana, Eve ]
role : [ Developer, Designer, Manager, Developer, Designer ]
active : [ true, false, true, false, true ]
joined_dt : [ 2026-05-12, 2024-11-03, 2026-05-20, 2025-08-15, 2026-02-28 ]
```

Disk layout:
[id col][name col][role col][active col][joined_dt col]

To compute "avg active rate" the reader opens ONLY the active column – skips the other four entirely (projection pushdown).

2. Three speedups columnar gives you

- (i) Projection pushdown
Query asks for 2 of 6 columns → read 2 columns. Row-major has to scan every byte of every row.
- (ii) Better compression
A column has values of the same type and often the same shape (e.g. all roles are 1 of 3 strings). Columns compress much better than mixed-type rows.
- (iii) Predicate pushdown via min/max stats
Each row group stores min & max for every column. A query like "joined_date >= 2026-05-01" SKIPS row groups whose max is below that. See §4.

3. Compaction techniques — what lives INSIDE a column chunk

Dictionary encoding (low-cardinality columns)

Raw column:
role = ["Developer", "Designer", "Manager", "Developer", "Designer"]

Dictionary: { 0: "Developer", 1: "Designer", 2: "Manager" }
Codes: [0, 1, 2, 0, 1]

Each string is stored ONCE; the column itself becomes tiny integers.

Run-length encoding (long runs of the same value)

Raw column:
active = [true, true, true, true, false, false]

RLE:
[(true, 4), (false, 2)]

Plus general-purpose compression on top: Snappy (fast), ZSTD (balanced), GZIP (smaller). Default for Parquet: Snappy.

4. Storing a range — row groups + min/max stats let you SKIP data

Row Group 1
rows 1–1000

min(joined_date) = 2023-01-12
max(joined_date) = 2023-12-30

Row Group 2
rows 1001–2000

min(joined_date) = 2024-01-02
max(joined_date) = 2024-12-29

Row Group 3
rows 2001–3000

min(joined_date) = 2026-05-01
max(joined_date) = 2026-05-31

QUERY
WHERE joined_date >= '2026-05-01'

skip Row Group 1 (max < 2026-05-01)
skip Row Group 2 (max < 2026-05-01)
READ Row Group 3

Often 1/10th the I/O of a full scan.